

UART6 异常包与 CPU2/CPU3 通信影响分析

1. 问题背景

现场日志中出现较多 UART6 通信失败，例如：

```
液位稳定,电机不动作      液位跟随      液位值为300580 (0.1mm)液位跟随      [UART6] 通信失败, 重试 1/3
```

该日志容易被理解为“液位稳定状态下直接报 UART6 错误”。结合当前代码看，前半段液位日志没有换行，后续下一轮液位跟随读取频率时打印 UART6 重试信息，因此日志被粘在同一行。实际触发点通常是下一轮 `DSM_Get_LevelMode_Frequency()` 读取液位频率失败。

本分析关注两个问题：

- UART6 传感器通信为什么可能出现短帧、校验失败或超时。
- CPU2 与 CPU3 之间的 UART5 通信是否会占用 CPU 超过 1ms，并影响 UART6 轮询收包。

2. 当前通信结构

2.1 UART6 传感器通信

CPU2 的 USART6 当前配置为 9600 8N1：

- `LTD_MAIN_CPU2/Core/Src/usart.c`
- `MX_USART6_UART_Init()`

DSM 传感器通信使用前台轮询逐字节接收：

```
HAL_UART_Receive(&huart6, &byte, 1, 1)
```

涉及文件：

- `LTD_MAIN_CPU2/Services/Sensor/dsm_sensor_communication.c`
- `LTD_MAIN_CPU2/Services/Sensor/ltd_sensor_communication.c`
- `LTD_MAIN_CPU2/Services/Sensor/wireless_communication.c`

9600 波特率、8N1 下，1 字节线上时间约：

```
10 bit / 9600 = 1.04 ms
```

因此 UART6 的前台轮询对 1ms 级中断占用比较敏感。若 CPU 长时间停留在其他中断中，主循环无法及时调用 `HAL_UART_Receive()` 读取 USART6 数据，可能造成：

- 收不到数据，`recvLen == 0`。
- 收到不完整短帧，`recvLen < 3` 或业务字段不完整。
- 中间丢字节导致 BCC 校验失败。
- 硬件 ORE 过载错误，但当前日志没有单独打印 ORE 原因。

2.2 CPU2/CPU3 通信

CPU2 与 CPU3 使用 UART5，115200 8N1，DMA 收发：

- `LTD_MAIN_CPU2/Core/Src/usart.c`
- `MX_UART5_Init()`

UART5 DMA 只能降低搬运字节的 CPU 占用，不代表一帧结束后的业务处理不占 CPU。CPU2 的 UART5 IDLE 中断里直接调用了 Modbus 处理：

```
HostCommuProcess(UART5_RX_BUF, UART5_RX_LEN);
```

涉及文件：

- `LTD_MAIN_CPU2/Core/Src/stm32f4xx_it.c`
- `LTD_MAIN_CPU2/Services/Modbus/hostcommu.c`
- `LTD_MAIN_CPU2/Services/Modbus/hostcommu_modbus.c`
- `LTD_MAIN_CPU2/Services/Modbus/dataanalysis_modbus.c`

因此 UART5 虽然使用 DMA 收发，但 CPU2 在 UART5 中断中仍会执行完整 Modbus 解析、寄存器映射、CRC 计算、组包和启动 DMA 发送。

3. HostCommuProcess 耗时分析

CPU2 当前系统时钟约 150 MHz：

```
1 ms = 150000 cycles
```

`HostCommuProcess()` 的耗时取决于 Modbus 功能码和异常路径。

3.1 正常 03 读保持寄存器

主要路径：

```
HostCommuProcess()  
-> SlaveCheckCRC()  
-> UpdateRcvPara()  
-> Response03Process()  
    -> WriteDeviceParamsToHoldingRegisters()  
    -> Compose03Package()  
        -> ReadRegister()
```

```
-> CRC16_Calculate()
-> HAL_UART_Transmit_DMA()
```

`WriteDeviceParamsToHoldingRegisters()` 当前会写出约 123 个 32 位参数，每个参数拆成两个 16 位保持寄存器。该路径没有大量阻塞外设访问，正常估算约 $0.1\sim0.4$ ms，一般不应超过 1ms。

3.2 正常 04 读输入寄存器

主要路径：

```
HostCommuProcess()
-> SlaveCheckCRC()
-> UpdateRcvPara()
-> Response04Process()
    -> write_measurement_result_to_InputRegisters()
    -> Compose04Package()
        -> ReadRegister()
-> CRC16_Calculate()
-> HAL_UART_Transmit_DMA()
```

`write_measurement_result_to_InputRegisters()` 中有几个明显耗时点：

- `memset()` 清零 `INPUTREGISTER_AMOUNT` 个输入寄存器，当前输入寄存器数量约 1304。
- 写入测量结果和调试数据。
- 循环写入最多 `MAX_MEASUREMENT_POINTS = 100` 个密度点，每点 6 个 32 位字段。
- 调用 `MotorCtrl_GetDisplayState()` 读取电机显示状态。

其中 `MotorCtrl_GetDisplayState()` 会通过 SPI2 读取 TMC5130 `RAMPSTAT`。SPI2 当前预分频为 64，PCLK1 约 37.5 MHz，SPI 时钟约 586 kHz。一次普通 TMC5130 寄存器读取需要两帧 SPI，每帧 5 字节，并带 CS guard 延时，估算约 $0.16\sim0.3$ ms。

因此 04 正常路径整体估算约 $0.4\sim0.9$ ms。在以下情况下可能超过 1ms：

- CPU3 单次读取寄存器数量较多。
- TMC5130 读取失败，进入额外推断读取路径。
- 同时有其他高优先级中断打断。
- 编译优化较低或调试打印未关闭。

3.3 0x10 写保持寄存器

0x10 路径风险最高：

```
HostCommuProcess()
-> Response10Process()
    -> WriteDeviceParamsToHoldingRegisters()
    -> Compose10Package()
        -> PresetRegister()
    -> ReadDeviceParamsFromHoldingRegisters()
```

```
-> MotorCtrl_SetCurrent()  
-> MotorCtrl_ApplyPositionSourceParams()  
-> request_device_params_save()  
-> CRC16_Calculate()  
-> HAL_UART_Transmit_DMA()
```

当前 `PresetRegister()` 内存在未受调试宏保护的打印：

```
printf("HoldingRegisterArray[%d] = %d\n", i, HoldingRegisterArray[i]);
```

`Response10Process()` 内也有打印：

```
printf("command: %lu\r\n", (unsigned long)g_deviceParams.command);
```

CPU2 的 `printf` 最终通过 USART1 阻塞发送：

```
HAL_UART_Transmit(&huart1, (uint8_t*) &ch, 1, HAL_MAX_DELAY);
```

USART1 为 115200 波特率，1 字节约 86.8 us。若一行打印约 30 字节，仅串口发送就约 2.6 ms。写多个寄存器时，`PresetRegister()` 会每个寄存器打印一行，因此 0x10 路径非常容易超过 1ms。

3.4 CRC 错误帧

CRC 错误路径也存在未受调试宏保护的打印：

- `SlaveCheckCRC()` 打印计算 CRC 和帧 CRC。
- `HostCommuProcess()` 再次打印收到的原始帧字节。

如果 CPU2/CPU3 通信线上偶发 CRC 错误，UART5 中断内会进行多行阻塞打印。该路径可能占用数 ms 到十几 ms，是影响 UART6 轮询收包的高风险路径。

4. UART5 对 UART6 的影响链路

影响链路如下：

```
CPU3 发送 UART5 Modbus 请求
-> CPU2 UART5 DMA 收包
-> UART5 IDLE 中断触发
-> CPU2 在中断中执行 HostCommuProcess()
-> 主循环暂时无法继续轮询 USART6
-> 传感器此时返回 UART6 数据
-> USART6 数据未及时读取
-> 可能出现短帧、超时、BCC 错误或 ORE
-> 打印 [UART6] 通信失败, 重试 x/3
```

关键点:

- UART5 DMA 只负责搬运 UART5 字节。
- `HostCommuProcess()` 仍在 UART5 中断上下文中执行。
- USART6 没有使用 DMA 或接收中断, 而是前台 1ms 轮询。
- USART6 9600 波特率下 1 字节约 1.04ms, 和当前轮询超时时间同量级。

因此只要 UART5 中断内处理超过约 1ms, 就可能提高 UART6 收包异常概率。若超过 2ms, 风险更明显。

5. 校验码与短帧问题

当前 DSM 响应有 BCC 校验, 但这不能完全阻止“短帧被当成有效数值”。

原因是:

- 当前收包主要检查 `recvLen == 0`、`recvLen < 3` 和 BCC。
- 频率解析函数会跳过前导字符, 找到第一个数字后用 `strtod()` 解析。
- 没有严格校验帧类型、固定长度、数字位数和结束符组合。

因此某些短帧只要自身 BCC 能对上, 仍可能被解析成 51~59 Hz 这类异常频率。例如现场看到的短帧形态:

```
F00054G
F00052A
F00051B
F00058K
```

这类帧并不一定是“完整频率帧”, 但当前解析逻辑可能把它们解释成 54 Hz、52 Hz、51 Hz、58 Hz。结论是: 有校验码仍然需要协议层做固定格式校验。

6. 与电机通信的关系

电机通信本身使用 SPI2/TMC5130, 与 UART6 不是同一个外设, 理论上没有直接串口资源冲突。

但当前有三个间接影响:

1. CPU2 处理 04 输入寄存器时会调用 `MotorCtrl_GetDisplayState()`, 该函数会读 TMC5130。也就是说, CPU3 查询输入寄存器时, UART5 中断内可能包含一次电机 SPI 读操作。

2. UART5 IDLE 中断会抢占主循环或测量流程中正在执行的电机控制代码。当前 TMC5130 读写使用阻塞式 `HAL_SPI_TransmitReceive()` / `HAL_SPI_Transmit()`，若 UART5 中断在一次 TMC5130 SPI 帧、两帧流水线读取之间，或电机等待到位轮询过程中插入，电机通信会被暂停或拉长。
3. 电机运行时可能带来电源、地线、EMI 干扰，影响 UART6 物理层质量，也可能影响 TMC5130 SPI 物理层稳定性。

所以“电机通信会被中断干扰”本身不直接等同于 UART6 一定异常，但在当前架构下，UART5 中断内的长耗时会同时影响两类前台通信：

- 对 UART6：主循环不能及时轮询接收传感器回包。
- 对电机：前台 TMC5130 SPI 通信和电机状态轮询会被中断打断，通信过程被拉长，严重时可能增加 TMC5130 读写失败、运动状态判断滞后、等待到位超时或异常停机概率。

6.1 对正在运行电机通信的影响链路

当前电机运行、等待到位、位置刷新和状态判断过程中会多次访问 TMC5130，例如：

- `MotorCtrl_GetDisplayState()` 读取 `RAMPSTAT`。
- `MotorDriver_InferDisplayStateFromDriver()` 读取 `VACTUAL` / `XACTUAL` / `XTARGET`。
- `MotorCtrl_UpdateDrumStateFromXActual()` 读取 `XACTUAL`。
- `stpr_waitMove()` 等待运动完成时读取 `RAMPSTAT` / `XACTUAL` / `XTARGET`。

TMC5130 寄存器读取存在流水线特性，一次逻辑读取通常要发送两帧 SPI。`XACTUAL` 为了过滤跳变，还可能连续读取两到三次。若 UART5 中断在这些访问过程中插入，可能产生以下影响：

电机控制流程正在读写 TMC5130

- > UART5 IDLE 中断触发
- > CPU2 进入 `HostCommuProcess()`
- > 中断内执行 Modbus 解析、寄存器映射、CRC、`printf` 或 TMC5130 状态读取
- > 前台 TMC5130 SPI 通信被暂停
- > 单次电机通信耗时被拉长，连续轮询节奏被打乱
- > 可能出现 TMC5130 读写失败、状态判断滞后、等待到位超时

这里的风险不是 SPI2 和 UART5 外设互相占用，而是 **UART5 中断占用 CPU，打断了前台阻塞式 SPI 通信流程**。尤其在 0x10 写寄存器、CRC 错误打印、CPU3 高频 04 轮询、TMC5130 读取失败回退等场景下，UART5 中断占用时间更长，对电机通信影响更明显。

7. 结论

1. UART5 使用 DMA 收发，但 CPU2 当前在 UART5 IDLE 中断里直接执行完整 `HostCommuProcess()`，因此仍可能长时间占用 CPU。
2. 正常 03 读保持寄存器大概率低于 1ms。
3. 正常 04 读输入寄存器估算约 `0.4~0.9ms`，在 TMC5130 额外读取或寄存器数量较多时可能超过 1ms。
4. 0x10 写保持寄存器和 CRC 错误帧路径存在阻塞 `printf`，非常容易超过 1ms。
5. USART6 当前是 9600 波特率前台 1ms 轮询收包，容易受到 1ms 级中断占用影响。
6. TMC5130 电机通信当前也是前台阻塞式 SPI 访问，UART5 中断长时间占用 CPU 会打断正常运行中的电机通信和状态轮询。
7. UART6 有 BCC 校验，但当前缺少严格帧格式校验，短帧仍可能通过 BCC 并被解析成 51~59 Hz。

综合判断：当前 UART6 异常包增多，既可能来自传感器物理层干扰，也可能被 UART5 中断内耗时放大。同时，该问题也会影响电机通信：当电机正在运行、等待到位或刷新位置时，UART5 中断长时间占用 CPU 会打断 TMC5130 SPI 读写节奏。尤其在 CPU3 高频轮询、写参数、CRC 错误、TMC5130 读状态失败或电机运行干扰同时出现时，UART6 短帧、TMC5130 通信失败和电机状态判断异常的概率都会升高。

8. 建议验证

8.1 测量 HostCommuProcess 实际耗时

推荐用 GPIO 翻转或 DWT 计数测量，不建议在中断内继续 `printf`。

测量点：

```
// UART5_IRQHandler 中，调用 HostCommuProcess 前后  
HostCommuProcess(UART5_RX_BUF, UART5_RX_LEN);
```

建议分别统计：

- 03 读保持寄存器最大耗时。
- 04 读输入寄存器最大耗时。
- 0x10 写保持寄存器最大耗时。
- CRC 错误帧最大耗时。

判断标准：

- `<0.5ms`：对 UART6 风险较低。
- `0.5~1ms`：有边界风险。
- `>1ms`：对 USART6 9600 轮询收包有明显风险。
- `>2ms`：很可能造成 UART6 丢字节或短帧。

8.2 增加 UART6 错误原因计数

当前 [UART6] 通信失败，重试 `x/3` 信息过于笼统，建议区分：

- 发送失败。
- `recvLen == 0`。
- `recvLen < 3`。
- BCC 校验失败。
- 设备错误响应。
- USART6 ORE/FE/NE 标志。
- 收到的原始 HEX 帧。

这样可以判断现场主要是没收到、短帧、校验错，还是硬件过载错误。

8.3 做 CPU3 轮询频率对比实验

建议测试三组现场数据：

1. CPU3 正常轮询。

2. CPU3 降低轮询频率。
3. CPU3 暂停轮询，只保留 CPU2 与 UART6 传感器通信。

如果 UART6 异常包随 CPU3 轮询频率下降明显减少，说明 UART5 中断占用或 RS485 通信负载是重要因素。

8.4 做电机运行相关性实验

建议对比：

- 电机完全不动作。
- 电机低速动作。
- 电机高速动作。
- 电机启停瞬间。

同时记录 UART6 异常类型和原始帧。如果异常集中在启停或高负载动作阶段，需重点检查电机供电、地线、屏蔽、传感器线缆走线和 UART6 接收错误标志。

同时建议记录电机侧指标：

- TMC5130 读写失败次数。
- `stpr_waitMove()` 等待到位耗时和超时次数。
- `XACTUAL` / `XTARGET` / `RAMPSTAT` / `GSTAT` 异常读数。
- CPU3 轮询开启和暂停两种情况下的电机通信失败率。

如果 CPU3 轮询或 0x10 写参数期间，电机通信失败率明显升高，说明 UART5 中断内处理对电机 SPI 通信也有实际影响。

9. 建议修改方向

9.1 解耦 UART5 中断和 Modbus 业务处理

长期建议：

- UART5 IDLE 中断只做 DMA 停止、长度记录、复制数据或置位事件。
- `HostCommuProcess()` 移到主循环或通信任务中执行。
- 中断内禁止业务处理和阻塞打印。

这样可以从根本上降低 UART5 对 UART6 前台轮询和 TMC5130 前台 SPI 电机通信的影响。

9.2 移除中断路径内阻塞 printf

至少应先处理：

- CRC 错误路径中的打印。
- `PresetRegister()` 每个寄存器一行的打印。
- `Response10Process()` 中的 `command` 打印。
- `MotorCtrl_ApplyPositionSourceParams()` 中可能被 0x10 路径触发的打印。

建议改为计数、事件标志或主循环异步打印。

9.3 避免 04 读输入寄存器时实时访问 TMC5130

`write_measurement_result_to_InputRegisters()` 中不要在 Modbus 中断路径实时读 TMC5130。建议：

- 电机状态由主循环周期性刷新到 `g_measurement.debug_data.motor_state`。
- 04 响应只读取缓存值。

这样可以避免 CPU3 每次读输入寄存器都触发一次电机 SPI 访问，也避免 UART5 中断内和前台电机控制流程同时竞争 TMC5130 访问时序。

9.4 加强 UART6 DSM 帧格式校验

对液位频率响应建议增加严格校验：

- 帧头必须符合协议。
- 数字位数必须符合协议。
- BCC 位置必须固定。
- 结束符必须符合协议。
- 频率范围必须合理，短帧不能直接进入 `strtod()`。

校验失败时应返回明确错误，不更新液位频率和液位判断状态。

9.5 考虑 UART6 使用 DMA 或中断接收

如果 UART6 传感器响应对时序敏感，长期建议将 USART6 从前台轮询改为：

- DMA + IDLE 接收。
- 或接收中断 + 环形缓冲。

这样即使主循环短时间被中断打断，也不容易丢失 UART6 字节。

10. 优先级建议

短期优先：

1. 给 UART6 错误增加原因计数和原始帧 HEX。
2. 用 GPIO 或 DWT 测量 `HostCommuProcess()` 最大耗时。
3. 增加 TMC5130 读写失败、等待到位超时和异常读数计数，和 CPU3 轮询状态一起记录。
4. 移除 UART5 中断路径内 0x10 和 CRC 错误的阻塞打印。
5. 给 DSM 频率响应加固定格式校验，避免短帧被解析成有效频率。

中期优先：

1. 将 `HostCommuProcess()` 从 UART5 IRQ 移到主循环。
2. 04 输入寄存器响应只读缓存，不在响应时读 TMC5130。
3. 将 UART6 接收改为 DMA/IDLE 或中断环形缓冲。